

SEED Labs: MD5 Collision Attack Lab

Task 2: Understanding MD5's Property & Suffix Extension Vulnerability

1. Overview & Core Property

In this task, we demonstrate a fundamental structural property of the MD5 hashing algorithm (and other Merkle–Damgård hash constructions):

Mathematical Property:

Given two distinct inputs M and N that form an MD5 collision (i.e., $MD5(M) = MD5(N)$), appending the *exact same suffix* T to both files results in two updated inputs that also collide:

$$MD5(M \parallel T) = MD5(N \parallel T)$$

2. Crucial Correction: How M and N Must Be Created

⚠ CRITICAL POINT: You CANNOT use arbitrary custom text files for M and N

If you manually create two random text files (e.g., `file1.txt` with "hello" and `file2.txt` with "world"), their initial hash values will **NOT** be equal ($MD5(M) \neq MD5(N)$).

Appending the same suffix T to non-colliding files will **never** produce identical hashes. Therefore, M and N **must** be a pair of colliding files generated via a collision tool like `fastcoll` (or generated in Task 1).

3. Step-by-Step Practical Procedure & Commands

Step 1: Generate the Initial Colliding Files (M and N)

Use `fastcoll` to generate two binary files M and N that have identical MD5 hash values:

```
# Generate prefix collision files prefix_out1.bin (M) and prefix_out2.bin (N)
fastcoll -o prefix_out1.bin prefix_out2.bin
```

Verify that both files share the exact same hash:

```
md5sum prefix_out1.bin prefix_out2.bin
```

Step 2: Create the Common Suffix File (T)

Now, you can create file T with **any custom content/text** you like:

```
# Create suffix file T with arbitrary content
echo "This is the common suffix data T appended to both files." > suffix_T.txt
```

Step 3: Concatenate T to M and N

Use the Linux `cat` command to append suffix T to file M ($M \parallel T$) and to file N ($N \parallel T$):

```
# Append T to M (prefix_out1.bin) -> output M_new.bin
cat prefix_out1.bin suffix_T.txt > M_new.bin

# Append T to N (prefix_out2.bin) -> output N_new.bin
cat prefix_out2.bin suffix_T.txt > N_new.bin
```

Step 4: Verify the Hashes of the Combined Files

Check the MD5 hashes of the new files:

```
md5sum M_new.bin N_new.bin
```

Result: Both `M_new.bin` and `N_new.bin` will output the **exact same MD5 hash value**, proving the property!

4. Theoretical Explanation (Why It Works)

MD5 uses an iterative compression function f :

1. The compression function takes an Intermediate Hash Value (IHV) and a 64-byte message block.
2. Since $MD5(M) = MD5(N)$, processing M and N leaves the algorithm in the exact same internal state:

$$IHV_M = IHV_N$$

3. When appending T , MD5 continues computing from IHV_M (or IHV_N) over the block(s) of T .
4. Since both computation paths start from identical intermediate states and consume identical block inputs, the final output state IHV_{final} must be identical.